ML Loop Code-Along - Step 11

Pipeline: make the safe path the easy path

Incremental prerequisite prep for Imperial ML/AI coursework

Prepared for Anthony Watts - June 2026

Rule: nothing appears before it is explained.

11. Pipeline: make the safe path the easy path

Manual scaling is educational, but easy to mess up. A Pipeline bundles preprocessing and modelling so the same sequence is applied consistently.

Step 11.1: Import the pipeline helper

Why this cell exists
make_pipeline builds a sequence of steps. During fitting, it fits preprocessing on training data and then fits the model.

```
from sklearn.pipeline import make_pipeline
```

What to look for
This is a safety tool as much as a convenience tool.

Step 11.2: Build a scaled KNN pipeline

Why this cell exists
The pipeline says: first standardise the features, then run KNN. This keeps preprocessing attached to the model workflow.

```
knn_pipeline = make_pipeline(
    StandardScaler(),
    KNeighborsClassifier(n_neighbors=5),
)
```

What to look for
This object has not learned yet. It is just the recipe.

Step 11.3: Fit and evaluate the pipeline

Why this cell exists
The pipeline receives raw training data. Internally it fits the scaler on training data, transforms the training data, then fits KNN.

```
knn_pipeline.fit(X_cls_train, y_cls_train)

pipeline_predictions = knn_pipeline.predict(X_cls_test)

print("Pipeline KNN accuracy:", round(
    accuracy_score(y_cls_test, pipeline_predictions), 3
))
```

What to look for
This should be close to the manual scaled KNN result, but with less chance of train/test leakage.